

# AI Operations, Module 1 Assignment

Raghava Roll No: DA24B021 August 30, 2026

## Question 1: Technical Debt Diagnosis

- (a) **Entanglement (CACE).** The model does not keep its features in separate boxes. Rounding the delivery time differently shifted that input, and when the model retrained it leaned on everything else a bit differently, which is how a feature nobody touched ended up worse.
- (b) **Undeclared Consumers.** Another team started reading the output table and never said so. The ML team is now on the hook for an interface it does not know about, and cannot change anything without risking breaking it.
- (c) **Configuration and Glue-Code Debt.** The settings and the order the scripts run in only exist in the head of whoever wrote them. There is no single file to look at, so nobody else can run the pipeline, or say what actually produced any given model.

**A fix for (c).** Rewrite the fourteen scripts as a DVC pipeline. Every step gets an entry in `dvc.yaml` saying what it reads and what it writes, and the hyperparameters all move into one `params.yaml`. The pipeline then documents itself, `dvc repro` only reruns the steps whose inputs actually changed, and a single commit pins the whole thing rather than just the code.

This quietly helps with (a) too. Once the delivery-time rounding is a declared parameter with declared downstream steps, changing it turns up as a diff and everything depending on it gets rerun, so the entanglement shows itself before anything ships instead of appearing as a mystery regression a fortnight later.

## Question 2: MLflow Experiment Comparison

I ran six experiments on a 20,000-image subset of MNIST using an `MLPClassifier`, 25 epochs each, varying the learning rate (0.0001, 0.001, 0.01) and the batch size (32, 256).

`mlp-lr0.001-bs32` came out on top at 0.9688 validation accuracy, hit on epoch 19. It also happened to have the lowest training loss at 0.0095, so in this sweep the best fit and the best generalisation landed on the same run. I still ranked on validation accuracy rather than loss, because there is no reason those two should agree.

Two things point at overfitting. The best run peaked at 0.9688 on epoch 19 and finished at 0.9608, while `mlp-lr0.01-bs32` fell further, 0.9523 down to 0.9393. Both were already getting worse before training stopped. The generalisation gap says the same thing: it widened in all six runs while training loss carried on falling, 0.0130 to 0.0338 for `mlp-lr0.001-bs32` and 0.0038 to 0.0329 for `mlp-lr0.001-bs256`.

Learning rate mattered far more than batch size. Averaged over batch sizes, peak accuracy moved 0.0272 across the three learning rates against only 0.0068 across the two batch sizes, about four times the effect. The weakest run, `mlp-lr0.0001-bs256`, was still sitting at 0.2310 training loss after 25 epochs, so that one was undertrained rather than overfitted.

## Question 3: DVC Data Versioning

I tracked the manifest with DVC using an AWS S3 bucket as the remote, tagging `v1.0` at 1800 rows and `v2.0` at 2800 once `new-labels.zip` was added. The rollback made the mechanism obvious: after `git checkout v1.0` the file was still 2801 lines, and it only dropped back to 1801 after `dvc checkout`, because `git` moves the pointer and DVC goes and fetches the bytes.

## Question 4: Reproducibility Capstone

I was Partner B, so my job was to rebuild the run from the repository on its own. The metric came back at exactly 0.969200 test accuracy, and the dataset SHA256 matched the tag on Partner A's run, which confirms we trained on identical bytes.

Against a stated tolerance of  $\pm 0.005$  on `final_test_accuracy` the delta came out at 0.000000, and every logged parameter matched.